Kernel Debugging



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Preventing bugs



Static code analysis

- Static analysis can be run with the *sparse* tool
- *sparse* works with annotation and can detect various errors at compile time
  - Locking issues (unbalanced locking)
  - Address space issues, such as accessing user space pointer directly
- Analysis can be run using `make C=2` to run only on files that are recompiled
- Or with `make C=1` to run on all files
- Example of an unbalanced locking scheme:

```
rzn1_a5psw.c:81:13: warning: context imbalance in 'a5psw_reg_rmw' - wrong count
  at exit
```

SPARSE



Good practices in kernel development (1/2)

- When writing driver code, never expect the user to provide correct values. Always check these values.
- Use the WARN_ON() macro if you want to display a stacktrace when a specific condition did happen.
  - dump_stack() can also be used during debugging to show the current call stack.

```
static bool check_flags(u32 flags)
{
  if (WARN_ON(flags & STATE_INVALID))
    return -EINVAL;
  return 0;
}
```



Good practices in kernel development (2/2)

- If the values can be checked at compile time (configuration input, `sizeof`, structure fields), use the BUILD_BUG_ON() macro to ensure the condition is true.

```
BUILD_BUG_ON(sizeof(ctx->__reserved) != sizeof(reserved));
```

- If during compilation you have some warnings about unused variables/parameters, they must be fixed.
- Apply `checkpatch.pl --strict` when possible which might find some potential problems in your code.



Linux Kernel Debugging



Linux Kernel Debugging

- The Linux Kernel features multiple tools to ease kernel debugging:
  - A dedicated logging framework
  - A standard way to dump low level crash messages
  - Multiple runtime checkers to check for different kind of issues: memory issues, locking mistakes, undefined behaviors, etc.
  - Interactive or post-mortem debugging
- Many of those features need to be explicitly enabled in the kernel menuconfig, those are grouped in the `Kernel hacking -> Kernel debugging` menuconfig entry.
  - CONFIG_DEBUG_KERNEL should be set to "y" to enable other debug options.



Debugging using messages



Debugging/tracing using logs 1/4

- Good old printk()!
  - Works in all contexts
  - Can specify a log level ranging from `0` (emergency) to `7` (debug)
  - Be careful of the delays introduced when logs are printed out on a serial console at 115200 bauds
    - A `*_ratelimited()` version exists to limit the amount of print if called too often
  - Not recommended for upstream contributions

Example:

```
printk("in probe\n");
```

Here's what you get in the kernel log:

```
[    1.878382] in probe
```

All other logging facilities are based on it.



Debugging/tracing using logs 2/4

- The `pr_*()` family of functions
  - They include the log level in the name: pr_emerg(), pr_alert(), pr_crit(), pr_err(), pr_warn(), pr_notice(), pr_info(), pr_cont() and the special pr_debug() (see next pages)
  - They allow setting a manual prefix (eg. eases grepping):

```
#define pr_fmt(fmt) "foo: " fmt
```

- Also defined in include/linux/printk.h

Example:

```
pr_info("in probe\n");
```

Here's what you get in the kernel log:

```
[    1.878382] in probe
```

or similarly with a manual format:

```
[    1.878382] foo: in probe
```



Debugging/tracing using logs 3/4

- The `dev_*()` family of functions
  - They include a formatted standard prefix with the device name: dev_emerg(), dev_alert(), dev_crit(), dev_err(), dev_warn(), dev_notice(), dev_info() and the special dev_dbg() (see next pages)
  - They additionally take a pointer to struct device as first argument
  - Defined in include/linux/dev_printk.h
  - To be used in device drivers

Example:

```
dev_info(&pdev->dev, "in probe\n");
```

Here's what you get in the kernel log:

```
[    1.878382] serial 48024000.serial: in probe
[    1.884873] serial 481a8000.serial: in probe
```



Debugging/tracing using logs 4/4

- The kernel defines many more format specifiers than the standard `printf()` existing ones.
  - `%p`: Display the hashed value of pointer by default
  - `%px`: Always display the address of a pointer (use carefully on non-sensitive)
  - `%pK`: Display hashed pointer value, zeros or the pointer address depending on `kptr_restrict` sysctl value
  - `%pOF`: Device-tree node format specifier
  - `%pr`: Resource structure format specifier
  - `%pa`: Physical address display (work on all architectures 32/64 bits)
  - `%pe`: Error pointer (displays the string corresponding to the error number)
- See core-api/printk-formats for an exhaustive list of format specifiers
- Also features a helper to dump entire buffers with a `hexdump` like display: print_hex_dump()



pr_debug() and dev_dbg()

- When the driver is compiled with `DEBUG` defined, all these messages are compiled and printed at the debug level. `DEBUG` can be defined at the beginning of the driver, or using `ccflags-$(CONFIG_DRIVER) += -DDEBUG` in the `Makefile`
- When the kernel is compiled with CONFIG_DYNAMIC_DEBUG, then these messages can dynamically be enabled on a per-file, per-module or per-message basis, by writing commands to `/proc/dynamic_debug/control`. Note that messages are not enabled by default.
  - Details in admin-guide/dynamic-debug-howto
  - Very powerful feature to only get the debug messages you're interested in.
- When neither `DEBUG` nor CONFIG_DYNAMIC_DEBUG are used, these messages are not compiled in.



pr_debug() and dev_dbg() usage

- Debug prints can be enabled using the `/proc/dynamic_debug/control` file.
  - `cat /proc/dynamic_debug/control` will display all lines that can be enabled in the kernel
  - Example: `init/main.c:1427 [main]run_init_process =p " %s"`
- A syntax allows to enable individual print using lines, files or modules
  - `echo "file drivers/pinctrl/core.c +p" > /proc/dynamic_debug/control` will enable all debug prints in `drivers/pinctrl/core.c`
  - `echo "module pciehp +p" > /proc/dynamic_debug/control` will enable the debug print located in the `pciehp` module
  - `echo "file init/main.c line 1427 +p" > /proc/dynamic_debug/control` will enable the debug print located at line 1247 of file `init/main.c`
  - Replace `+p` with `-p` to disable the debug print



Debug logs troubleshooting

- When using dynamic debug, make sure that your debug call is enabled: it must be visible in `control` file in debugfs **and** be activated (`=p`)
- Is your log output only in the kernel log buffer?
  - You can see it thanks to `dmesg`
  - You can lower the `loglevel` to output it to the console directly
  - You can also set `ignore_loglevel` in the kernel command line to force all kernel logs to console
- If you are working on an out-of-tree module, you may prefer to define `DEBUG` in your module source or Makefile instead of using dynamic debug
- If configuration is done through the kernel command line, is it properly interpreted?
  - Starting from 5.14, kernel will let you know about faulty command line: `Unknown kernel command line parameters foo, will be passed to user space.`
  - You may need to take care of special characters escaping (e.g: quotes)
- Be aware that a few subsystems bring their own logging infrastructure, with specific configuration/controls, eg: `drm.debug=0x1ff`



Kernel early debug

- When booting, the kernel sometimes crashes even before displaying the system messages
- On ARM, if your kernel doesn't boot or hangs without any message, you can activate early debugging options
  - CONFIG_DEBUG_LL=y to enable ARM early serial output capabilities
  - CONFIG_EARLY_PRINTK=y will allow printk to output the prints earlier
- `earlyprintk` command line parameter should be given to enable early printk output

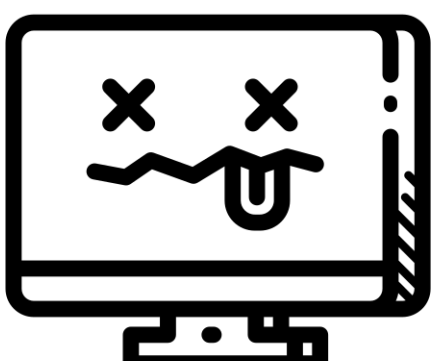


Kernel crashes and oops



Kernel crashes

- The kernel is not immune to crash, many errors can be done and lead to crashes
  - Memory access error (NULL pointer, out of bounds access, etc)
  - Voluntarily panicking on error detection (using panic())
  - Kernel incorrect execution mode (sleeping in atomic context)
  - Deadlocks detected by the kernel (Soft lockup/locking problem)
- On error, the kernel will display a message on the console that is called a “Kernel oops”



Icon by Peter van Driel, TheNounProject.com



Kernel oops (1/2)

- The content of this message depends on the architecture that is used.
- Almost all architectures display at least the following information:
  - CPU state when the oops happened
  - Registers content with potential interpretation
  - Backtrace of function calls that led to the crash
  - Stack content (last X bytes)
- Depending on the architecture, the crash location can be identified using the content of the PC registers (sometimes named IP, EIP, etc).
- To have a meaningful backtrace with symbol names use CONFIG_KALLSYMS=y which will embed the symbol names in the kernel image.



Kernel oops (2/2)

- Symbols are displayed in the backtrace using the following format:
  - `<symbol_name>+<hex_offset>/<symbol_size>`
- If the oops is not critical (taken in process context), then the kernel will kill process and continue its execution
  - The kernel stability might be compromised!
- Tasks that are taking too much time to execute and that are hung can also generate an oops (CONFIG_DETECT_HUNG_TASK)
- If KGDB support is present and configured, on oops, the kernel will switch to KGDB mode.



Oops example (1/2)

```
[    1.635909] 8<--- cut here ---
[    1.639034] Unable to handle kernel NULL pointer dereference at virtual address 00000050
[    1.647304] [00000050] *pgd=00000000
[    1.650959] Internal error: Oops: 5 [#1] SMP ARM
[    1.655635] Modules linked in:
[    1.658736] CPU: 0 PID: 28 Comm: kworker/u4:1 Not tainted 5.18.0-rc6-01642-g59dcbe02d20b #48
[    1.667261] Hardware name: Generic DT based system
[    1.672104] Workqueue: events_unbound deferred_probe_work_func
[    1.678029] PC is at miic_create+0xac/0x134
[    1.682280] LR is at _raw_spin_unlock_irqrestore+0x2c/0x34
[    1.687841] pc : [<804ba6e4>]    lr : [<806811f0>]    psr: 20000053
[    1.694172] sp : 90a99cf8  ip : 90a99cc8  fp : 90a99d1c
[    1.699453] r10: 8089339e  r9 : 808a5a18  r8 : 8fdb4284
[    1.704733] r7 : 82173000  r6 : 8fdb47d8  r5 : 82173010  r4 : 8fdb3ad8
[    1.711327] r3 : 00000000  r2 : 00000000  r1 : a0000053  r0 : 8fdb3ad8
[    1.717921] Flags: nzCv  IRQs on  FIQs off  Mode SVC_32  ISA ARM  Segment none
[    1.725220] Control: 10c5387d  Table: 8000406a  DAC: 00000051
[    1.731022] Register r0 information: non-slab/vmalloc memory
[    1.736754] Register r1 information: non-paged memory
[    1.741867] Register r2 information: NULL pointer
[    1.746629] Register r3 information: NULL pointer
[    1.751389] Register r4 information: non-slab/vmalloc memory
[    1.757113] Register r5 information: slab kmalloc-512 start 82173000 pointer offset 16 size 512
[    1.765943] Register r6 information: non-slab/vmalloc memory
[    1.771668] Register r7 information: slab kmalloc-512 start 82173000 pointer offset 0 size 512
[    1.780401] Register r8 information: non-slab/vmalloc memory
[    1.786125] Register r9 information: non-slab/vmalloc memory
[    1.791848] Register r10 information: non-slab/vmalloc memory
[    1.797661] Register r11 information: 2-page vmalloc region starting at 0x90a98000 allocated at kernel_clone+0xb4/0x2a4
[    1.808567] Register r12 information: 2-page vmalloc region starting at 0x90a98000 allocated at kernel_clone+0xb4/0x2a4
[    1.819468] Process kworker/u4:1 (pid: 28, stack limit = 0x(ptrval))
[    1.825889] Stack: (0x90a99cf8 to 0x90a9a000)
[    1.830304] 9ce0:                                                       00000005 ddf3fe37
[    1.838570] 9d00: 8259c840 8259c840 82173010 8fdb47d8 90a99dac 90a99d20 804bb7b8 804ba644
[    1.846834] 9d20: 00000000 90a99d34 00000002 8fdb46bc 00000000 8fdb4284 00000000 804f1c80
[    1.855099] 9d40: ffffffff 00000001 8089ad70 ffffffff 8fdb43f8 ff8fb728 ff8fb728 ddf3fe37
[    1.863364] 9d60: 82173010 82173010 81383ea0 82173010 00000002 8219cac0 90a99d9c ddf3fe37
[    1.871627] 9d80: 80458940 00000000 82173010 81383ea0 82173010 00000002 8219cac0 8200f00d
[    1.879892] 9da0: 90a99dcc 90a99db0 80449f44 804bb664 00000000 82173010 81383ea0 82173010
[    1.888155] 9dc0: 90a99dec 90a99dd0 80447cb8 80449ee8 82173010 81383ea0 81383ea0 82173010
[    1.896420] 9de0: 90a99e04 90a99df0 80447ef0 80447b9c 815b034c 815b0350 90a99e2c 90a99e08
[    1.904684] 9e00: 80447f54 80447e28 81383ea0 90a99e84 82173010 00000001 00000000 8219cac0
[    1.912948] 9e20: 90a99e4c 90a99e30 80448320 80447f10 00000000 90a99e84 8044827c 00000001
[    1.921213] 9e40: 90a99e7c 90a99e50 80445dac 80448288 82173010 82098e6c 82575ab8 ddf3fe37
[    1.929478] 9e60: 90a99e7c 82173010 8137c268 82173054 90a99eac 90a99e80 80448158 80445d00
[    1.937742] 9e80: 90a99eac 82173010 00000001 ddf3fe37 82173010 8137c268 82173010 8200f000
[    1.946006] 9ea0: 90a99ebc 90a99eb0 80448444 80448084 90a99edc 90a99ec0 80446c00 80448434
```

Cause and generic information

Register content and CPU state

Stack content



Oops example (2/2)

```
    2.036894] Backtrace:
    2.039380]  miic_create from a5psw_probe+0x160/0x3e0
    2.044530]  r6:8fdb47d8 r5:82173010 r4:8259c840
    2.049194]  a5psw_probe from platform_probe+0x68/0xb8
    2.054432]  r10:8200f00d r9:8219cac0 r8:00000002 r7:82173010 r6:81383ea0 r5:82173010
    2.062332]  r4:00000000
    2.064896]  platform_probe from really_probe+0x128/0x28c
    2.070383]  r7:82173010 r6:81383ea0 r5:82173010 r4:00000000
    2.076095]  really_probe from __driver_probe_device+0xd4/0xe8
    2.082018]  r7:82173010 r6:81383ea0 r5:81383ea0 r4:82173010
    2.087730]  __driver_probe_device from driver_probe_device+0x50/0xd0
    2.094260]  r5:815b0350 r4:815b034c
    2.097874]  driver_probe_device from __device_attach_driver+0xa4/0xc4
    2.104499]  r9:8219cac0 r8:00000000 r7:00000001 r6:82173010 r5:90a99e84 r4:81383ea0
    2.112309]  __device_attach_driver from bus_for_each_drv+0xb8/0xd0
    2.118677]  r7:00000001 r6:8044827c r5:90a99e84 r4:00000000
    2.124388]  bus_for_each_drv from __device_attach+0xe0/0x158
    2.130226]  r6:82173054 r5:8137c268 r4:82173010
    2.134881]  __device_attach from device_initial_probe+0x1c/0x20
    2.140977]  r7:8200f000 r6:82173010 r5:8137c268 r4:82173010
    2.146689]  device_initial_probe from bus_probe_device+0x38/0x90
    2.152869]  bus_probe_device from deferred_probe_work_func+0x90/0xa8
    2.159408]  r7:8200f000 r6:8137c1ec r5:8137c188 r4:82173010
    2.165121]  deferred_probe_work_func from process_one_work+0x1ac/0x278
    2.171839]  r7:8200f000 r6:82006800 r5:8137c1b0 r4:82180000
    2.177550]  process_one_work from process_scheduled_works+0x38/0x3c
    2.184011]  r10:90841e94 r9:81399b20 r8:8200681c r7:80d03d00 r6:82006800 r5:82180018
    2.191913]  r4:82180000
    2.194478]  process_scheduled_works from worker_thread+0x23c/0x2d4
    2.200840]  r5:82180018 r4:82180000
    2.204455]  worker_thread from kthread+0xd4/0xdc
    2.209254]  r9:82180000 r8:80131078 r7:821e2150 r6:821e2080 r5:821e2140 r4:8219cac0
    2.217065]  kthread from ret_from_fork+0x14/0x2c
    2.221842] Exception stack(0x90a99fb0 to 0x90a99ff8)
    2.226958] 9fa0:                                     00000000 00000000 00000000 00000000
    2.235220] 9fc0: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
    2.243479] 9fe0: 00000000 00000000 00000000 00000000 00000013 00000000
    2.250170]  r10:00000000 r9:00000000 r8:00000000 r7:00000000 r6:00000000 r5:801373c4
    2.258074]  r4:821e2140 r3:00000001
    2.261704] Code: e3500000 0affffe6 e3a03000 e1a00004 (e5936050)
    2.267944] ---[ end trace 0000000000000000 ]---
```

Backtrace

Exception stack content



Kernel oops debugging: addr2line

- In order to convert addresses/symbol name from this display to source code lines, one can use addr2line
  - `addr2line -e vmlinux <address>`
- GNU binutils >= 2.39 takes the symbol+offset notation too:
  - `addr2line -e vmlinux <symbol_name>+<off>`
- The symbol+offset notation can be used with older binutils versions via the `faddr2line` script in the kernel sources:
  - `scripts/faddr2line vmlinux <symbol_name>+<off>`
- The kernel must have been compiled with CONFIG_DEBUG_INFO=y to embed the debugging information into the vmlinux file.



Kernel oops debugging: `decode_stacktrace.sh`

- `addr2line` decoding of oopses can be automated using `decode_stacktrace.sh` script which is provided in the kernel sources.
- This script will translate all symbol names/addresses to the matching file/lines and will display the assembly code where the crash did trigger.
- ```
  ./scripts/decode_stacktrace.sh vmlinux [linux_source_path/] \
  < oops_report.txt > decoded_oops.txt
  ```
- NOTE: `CROSS_COMPILE` and `ARCH` env var should be set to obtain the correct disassembly dump.
  ```



Panic and oops behavior configuration

- Sometimes, crash might be so bad that the kernel will panic and halt its execution entirely by stopping scheduling application and staying in a busy loop.
- Automatic reboot on panic can be enabled via CONFIG_PANIC_TIMEOUT
  - 0: never reboots
  - Negative value: reboot immediately
  - Positive value: seconds to wait before rebooting
- OOPS can be configured to always panic:
  - at boot time, adding `oops=panic` to the command line
  - at build time, setting CONFIG_PANIC_ON_OOPS=y



Built-in kernel self tests



Kernel memory issue debugging

- The same kind of memory issues that can happen in user space can be triggered while writing kernel code
  - Out of bounds accesses
  - Use-after-free errors (dereferencing a pointer after `kfree()`)
  - Out of memory due to missing `kfree()`
- Various tools are present in the kernel to catch these issues
  - *KASAN* to find use-after-free and out-of-bound memory accesses
  - *KFENCE* to find use-after-free and out-of-bound in production systems
  - *Kmemleak* to find memory leak due to missing free of memory



KASAN

- Kernel Address Space Sanitizer
- Allows to find use-after-free and out-of-bounds memory accesses
- Uses GCC to instrument the kernel at compile-time
- Supported by almost all architectures (ARM, ARM64, PowerPC, RISC-V, S390, Xtensa and X86)
- Needs to be enabled at kernel configuration with CONFIG_KASAN
- Can then be enabled for files by modifying Makefile
  - `KASAN_SANITIZE_file.o := y` for a specific file
  - `KASAN_SANITIZE := y` for all files in the Makefile folder



Kmemleak

- Kmemleak allows to find memory leaks for dynamically allocated objects with `kmalloc()`
  - Works by scanning the memory to detect if allocated address are not referenced anymore anywhere (large overhead).
- Once enabled with CONFIG_DEBUG_KMEMLEAK, kmemleak control files will be visible in *debugfs*
- Memory leaks is scanned every 10 minutes
  - can be disabled via CONFIG_DEBUG_KMEMLEAK_AUTO_SCAN
- An immediate scan can be triggered using
  - `# echo scan > /sys/kernel/debug/kmemleak`
- Results are displayed in debugfs
  - `# cat /sys/kernel/debug/kmemleak`
- See dev-tools/kmemleak for more information



Kmemleak report

```
# cat /sys/kernel/debug/kmemleak
unreferenced object 0x82d43100 (size 64):
  comm "insmod", pid 140, jiffies 4294943424 (age 270.420s)
  hex dump (first 32 bytes):
    b4 bb e1 8f c8 a4 e1 8f 8c ce e1 8f 88 c6 e1 8f  ................
    10 a5 e1 8f 18 e2 e1 8f ac c6 e1 8f 0c c1 e1 8f  ................
  backtrace:
    [<c31f5b59>] slab_post_alloc_hook+0xa8/0x1b8
    [<c8200adb>] kmem_cache_alloc_trace+0xb8/0x104
    [<1836406b>] 0x7f005038
    [<89fff56d>] do_one_initcall+0x80/0x1a8
    [<31d908e3>] do_init_module+0x50/0x210
    [<2658dd55>] load_module+0x208c/0x211c
    [<e1d48f15>] sys_finit_module+0xe4/0xf4
    [<1de12529>] ret_fast_syscall+0x0/0x54
    [<7ee81f34>] 0x7eca8c80
```



UBSAN

- UBSAN is a runtime checker for code with undefined behavior
  - Shifting with a value larger than the type
  - Overflow of integers (signed and unsigned)
  - Misaligned pointer access
  - Out of bound access to static arrays
  - https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html
- It uses compile-time instrumentation to insert checks that will be executed at runtime
- Must be enabled using CONFIG_UBSAN=y
- Then, can be enabled for specific files by modifying Makefile
  - `UBSAN_SANITIZE_file.o := y` for a specific file
  - `UBSAN_SANITIZE := y` for all files in the Makefile folder



UBSAN: report example

- Report for an undefined behavior due to a shift with a value > 32.

```
UBSAN: Undefined behaviour in mm/page_alloc.c:3117:19
shift exponent 51 is too large for 32-bit type 'int'
CPU: 0 PID: 6520 Comm: syz-executor1 Not tainted 4.19.0-rc2 #1
Hardware name: QEMU Standard PC (i440FX + PIIX, 1996), BIOS Bochs 01/01/2011
Call Trace:
__dump_stack lib/dump_stack.c:77 [inline]
dump_stack+0xd2/0x148 lib/dump_stack.c:113
ubsan_epilogue+0x12/0x94 lib/ubsan.c:159
__ubsan_handle_shift_out_of_bounds+0x2b6/0x30b lib/ubsan.c:425
...
RIP: 0033:0x4497b9
Code: e8 8c 9f 02 00 48 83 c4 18 c3 0f 1f 80 00 00 00 00 48 89 f8 48
89 f7 48 89 d6 48 89 ca 4d 89 c2 4d 89 c8 4c 8b 4c 24 08 0f 05 <48> 3d
01 f0 ff ff 0f 83 9b 6b fc ff c3 66 2e 0f 1f 84 00 00 00 00
RSP: 002b:00007fb5ef0e2c68 EFLAGS: 00000246 ORIG_RAX: 0000000000000010
RAX: ffffffffffffffda RBX: 00007fb5ef0e36cc RCX: 00000000004497b9
RDX: 0000000020000040 RSI: 0000000000000258 RDI: 0000000000000014
RBP: 000000000071bea0 R08: 0000000000000000 R09: 0000000000000000
R10: 0000000000000000 R11: 0000000000000246 R12: 00000000ffffffff R13: 0000000000005490 R14: 00000000006ed530
R15: 00007fb5ef0e3700
```



Debugging locking

- Lock debugging: prove locking correctness
  - CONFIG_PROVE_LOCKING
  - Adds instrumentation to kernel locking code
  - Detect violations of locking rules during system life, such as:
    - Locks acquired in different order (keeps track of locking sequences and compares them).
    - Spinlocks acquired in interrupt handlers and also in process context when interrupts are enabled.
  - Not suitable for production systems but acceptable overhead in development.
  - See locking/lockdep-design for details
- CONFIG_DEBUG_ATOMIC_SLEEP allows to detect code that incorrectly sleeps in atomic section (while holding lock typically).
  - Warning displayed in `dmesg` in case of such violation.



Concurrency issues

- Kernel Concurrency SANitizer framework
- CONFIG_KCSAN, introduced in Linux 5.8.
- Dynamic race detector relying on compile time instrumentation.
- Can find concurrency issues (mainly data races) in your system.
- See dev-tools/kcsan and https://lwn.net/Articles/816850/ for details.

Practical lab - Kernel debugging



Debugging kernel programming mistakes with integrated frameworks

- Debug locking issues using lockdep
- Spot function calls in invalid context
- Use kmemleak to detect memory leaks on the system



The Magic SysRq



The Magic SysRq

Functionality provided by serial drivers

- Allows to run multiple debug/rescue commands even when the kernel seems to be in deep trouble
  - On embedded: in the console, send a break character (Picocom: press `[Ctrl] + a` followed by `[Ctrl] + \`), then press `<character>`
  - By echoing `<character>` in `/proc/sysrq-trigger`
- Example commands:
  - `h`: show available commands
  - `s`: sync all mounted filesystems
  - `b`: reboot the system
  - `w`: shows the kernel stack of all sleeping processes
  - `t`: shows the kernel stack of all running processes
  - `g`: enter kgdb mode
  - `z`: flush trace buffer
  - `c`: triggers a crash (kernel panic)
  - You can even register your own!
- Detailed in admin-guide/sysrq



KGDB



kgdb - A kernel debugger

- CONFIG_KGDB in *Kernel hacking*.
- The execution of the kernel is fully controlled by `gdb` from another machine, connected through a serial line.
- Can do almost everything, including inserting breakpoints in interrupt handlers.
- Feature supported for the most popular CPU architectures
- CONFIG_GDB_SCRIPTS allows to build GDB python scripts that are provided by the kernel.
  - See process/debugging/kgdb for more information



kgdb kernel config

- CONFIG_DEBUG_KERNEL=y to make KGDB support visible
- CONFIG_KGDB=y to enable KGDB support
- CONFIG_DEBUG_INFO=y to compile the kernel with debug info (`-g`)
- CONFIG_FRAME_POINTER=y to have more reliable stacktraces
- CONFIG_KGDB_SERIAL_CONSOLE=y to enable KGDB support over serial
- CONFIG_GDB_SCRIPTS=y to enable kernel GDB python scripts
- CONFIG_RANDOMIZE_BASE=n to disable KASLR
- CONFIG_WATCHDOG=n to disable watchdog
- CONFIG_MAGIC_SYSRQ=y to enable Magic SysReq support
- CONFIG_STRICT_KERNEL_RWX=n to disable memory protection on code section, thus allowing to put breakpoints



kgdb pitfalls

- KASLR should be disabled to avoid confusing gdb with randomized kernel addresses
  - Disable *kaslr mode using* `nokaslr` *command line parameter if enabled in your kernel.*
- Disable the platform watchdog to avoid rebooting while debugging.
  - When interrupted by KGDB, all interrupts are disabled thus, the watchdog is not serviced.
  - Sometimes, watchdog is enabled by upper boot levels. Make sure to disable the watchdog there too.
- Can not interrupt kernel execution from gdb using `interrupt` command or `Ctrl + C`.
- Not possible to break everywhere (see CONFIG_KGDB_HONOUR_BLOCKLIST).
- Need a console driver with polling support.
- Some architecture lacks functionalities (No watchpoints on arm32 for instance) and some instabilities might happen!



Using kgdb (1/2)

- Details available in the kernel documentation: process/debugging/kgdb
- You must include a kgdb I/O driver. One of them is `kgdb` over serial console (`kgdboc`: `kgdb` over console, enabled by CONFIG_KGDB_SERIAL_CONSOLE)
- Configure `kgdboc` at boot time by passing to the kernel:
  - `kgdboc=<tty-device>,<bauds>`.
  - For example: `kgdboc=ttyS0,115200`
- Or at runtime using sysfs:
  - `echo ttyS0 > /sys/module/kgdboc/parameters/kgdboc`
  - If the console does not have polling support, this command will yield an error.



Using kgdb (2/2)

- Then also pass `kgdbwait` to the kernel: it makes `kgdb` wait for a debugger connection.
- Boot your kernel, and when the console is initialized, interrupt the kernel with a break character and then `g` in the serial console (see our *Magic SysRq* explanations).
- On your workstation, start `gdb` as follows:
  - `arm-linux-gdb ./vmlinux`
  - `(gdb) set serial baud 115200`
  - `(gdb) target remote /dev/ttyS0`
- Once connected, you can debug a kernel the way you would debug an application program.
- On GDB side, the first threads represent the CPU context (ShadowCPU<x>), then all the other threads represents a task.



Kernel *GDB scripts*

- CONFIG_GDB_SCRIPTS allows to build a set of python script which ease the kernel debugging by adding new commands and functions.
- When using `gdb vmlinux`, the scripts present in vmlinux-gdb.py file at the root of build dir will be loaded automatically.
  - `lx-symbols`: (Re)load symbols for vmlinux and modules
  - `lx-dmesg`: display kernel dmesg
  - `lx-lsmod`: display loaded modules
  - `lx-device-bus|class|tree`: display device bus, classes and tree
  - `lx-ps`: `ps` like view of tasks
  - `$lx_current()` contains the current `task_struct` item `$lx_per_cpu(var, cpu)` returns a per-cpu variable
  - `apropos lx` To display all available functions.
- dev-tools/gdb-kernel-debugging



KDB

- CONFIG_KGDB_KDB includes a kgdb frontend name “KDB”
- This frontend exposes a debug prompt on the serial console which allows debugging the kernel without the need for an external gdb.
- KDB can be entered using the same mechanism used for entering kgdb mode.
- *KDB* and *KGDB* can coexist and be used at the same time.
  - Use the `kgdb` command in KDB to enter kgdb mode.
  - Send a maintenance packet from gdb using `maintenance packet 3` to switch from kgdb to KDB mode.



KDB commands

- KDB does not consume gdb commands but a set of dedicated KDB commands:
  - `go`: Continue execution
  - `bt`: Display backtrace
  - `env`: Show environment variables
  - `ps`: List all tasks
  - `pid`: Switch to another task
  - `md/mm`: Read/write memory
  - `lsmod`: List loaded modules
- To check all available commands, you can refer to the `help` command output, or check maintab in kernel source code

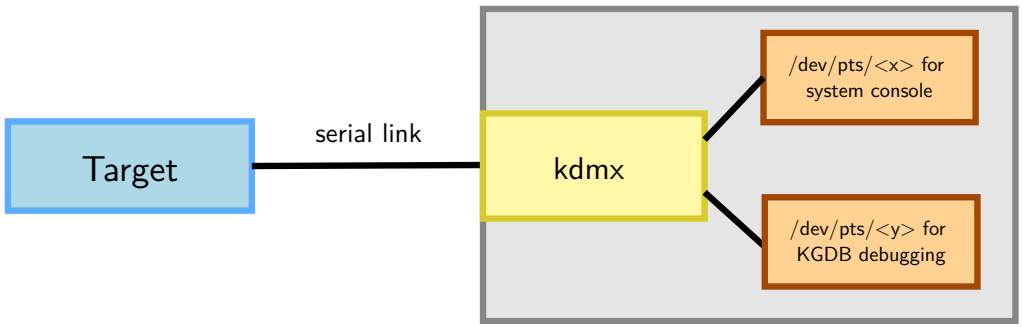


kdmx

- When the system has only a single serial port, it is not possible to use both KGDB and the serial line as an output terminal since only one program can access that port.
- Fortunately, the *kdmx* tool allows to use both KGDB and serial output by splitting GDB messages and standard console from a single port to 2 slave pty (`/dev/pts/x`)
- https://git.kernel.org/pub/scm/utils/kernel/kgdb/agent-proxy.git
  - Located in the subdirectory `kdmx`

```
$ kdmx -n -d -p/dev/ttyACM0 -b115200
serial port: /dev/ttyACM0
Initalizing the serial port to 115200 8n1
/dev/pts/6 is slave pty for terminal emulator
/dev/pts/7 is slave pty for gdb

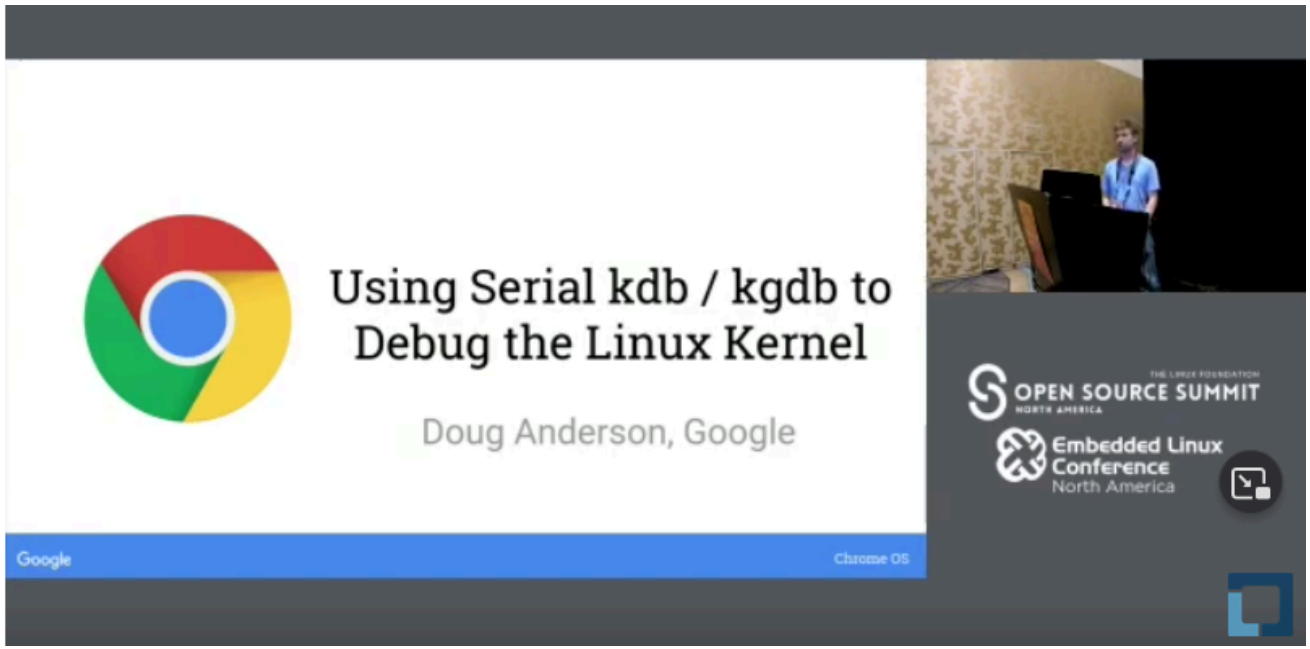
Use <ctrl>C to terminate program
```





Going further with KGDB

- Good presentation from Doug Anderson with a lot of demos and explanations
  - Video: https://www.youtube.com/watch?v=HBOwoSyRmys
  - Slides: https://elinux.org/images/1/1b/ELC19_Serial_kdb_kgdb.pdf





crash



crash

- *crash* is a CLI tool allowing to investigate kernel (dead or alive!)
  - Uses /dev/mem or /proc/kcore on live systems
  - Requires CONFIG_STRICT_DEVMEM=n
- Can use a coredump generated using kdump, kvmdump, etc.
- Based on `gdb` and provides many specific commands to inspect the kernel state.
  - Stack traces, dmesg (`log`), memory maps of the processes, irqs, virtual memory areas, etc.
- Allows examining all the tasks that are running on the system.
- Hosted at https://github.com/crash-utility/crash

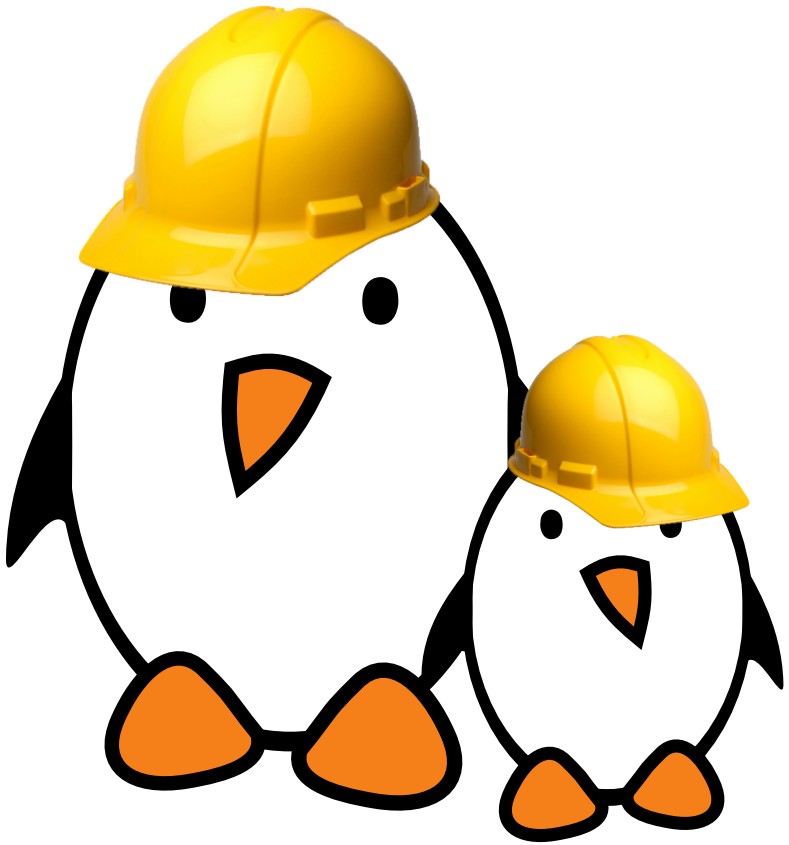


crash example

```
$ crash vmlinux vmcore
[...]
    TASKS: 75
NODENAME: buildroot
  RELEASE: 5.13.0
  VERSION: #1 SMP PREEMPT Tue Nov 15 14:42:25 CET 2022
  MACHINE: armv7l  (unknown Mhz)
  MEMORY: 512 MB
    PANIC: "Unable to handle kernel NULL pointer dereference at virtual address 00000070"
      PID: 127
  COMMAND: "watchdog"
    TASK: c3f163c0  [THREAD_INFO: c3f00000]
      CPU: 1
    STATE: TASK_RUNNING (PANIC)

crash> mach
    MACHINE TYPE: armv7l
     MEMORY SIZE: 512 MB
            CPUS: 1
 PROCESSOR SPEED: (unknown)
              HZ: 100
       PAGE SIZE: 4096
KERNEL VIRTUAL BASE: c0000000
KERNEL MODULES BASE: bf000000
KERNEL VMALLOC BASE: e0000000
KERNEL STACK SIZE: 8192
```

Practical lab - Kernel debugging



Debugging kernel crashes on a live kernel

- Analyze an OOPS message
- Debug a crash with KGDB



Post-mortem analysis



Kernel crash post-mortem analysis

- Sometimes, accessing the crashed system is not possible or the system can't stay offline while waiting to be debugged
- Kernel can generate crash dumps (a *vmcore* file) to a remote location, allowing to quickly restart the system while still be able to perform post-mortem analysis with GDB.
- This feature relies on *kexec* and *kdump* which will boot another kernel as soon as the crash occurs right after dumping the *vmcore* file.
  - The *vmcore* file can be saved on local storage, via SSH, FTP etc.



kexec & kdump (1/2)

- On panic, the kernel kexec support will execute a “dump-capture kernel” directly from the kernel that crashed
  - Most of the time, a specific dump-capture kernel is compiled for that task (minimal config with specific initramfs/initrd)
- *kexec* system works by saving some RAM for the kdump kernel execution at startup
  - `crashkernel` parameter should be set to specify the crash kernel dedicated physical memory region
- *kexec-tools* are then used to load dump-capture kernel into this memory zone using the `kexec` command
  - Internally uses the `kexec_load` system call man 2 kexec_load



kexec & kdump (2/2)

- Finally, on panic, the kernel will reboot into the "dump-capture" kernel allowing the user to dump the kernel coredump (`/proc/vmcore`) onto whatever media
- Additional command line options depends on the architecture
- See admin-guide/kdump/kdump for more comprehensive explanations on how to setup the kdump kernel with `kexec`.
- Additional user-space services and tools allow to automatically collect and dump the vmcore file to a remote location.
  - See kdump systemd service and the `makedumpfile` tool which can also compress the vmcore file into a smaller file (Only for x86, PPC, IA64, S390).
  - https://github.com/makedumpfile/makedumpfile

kdump

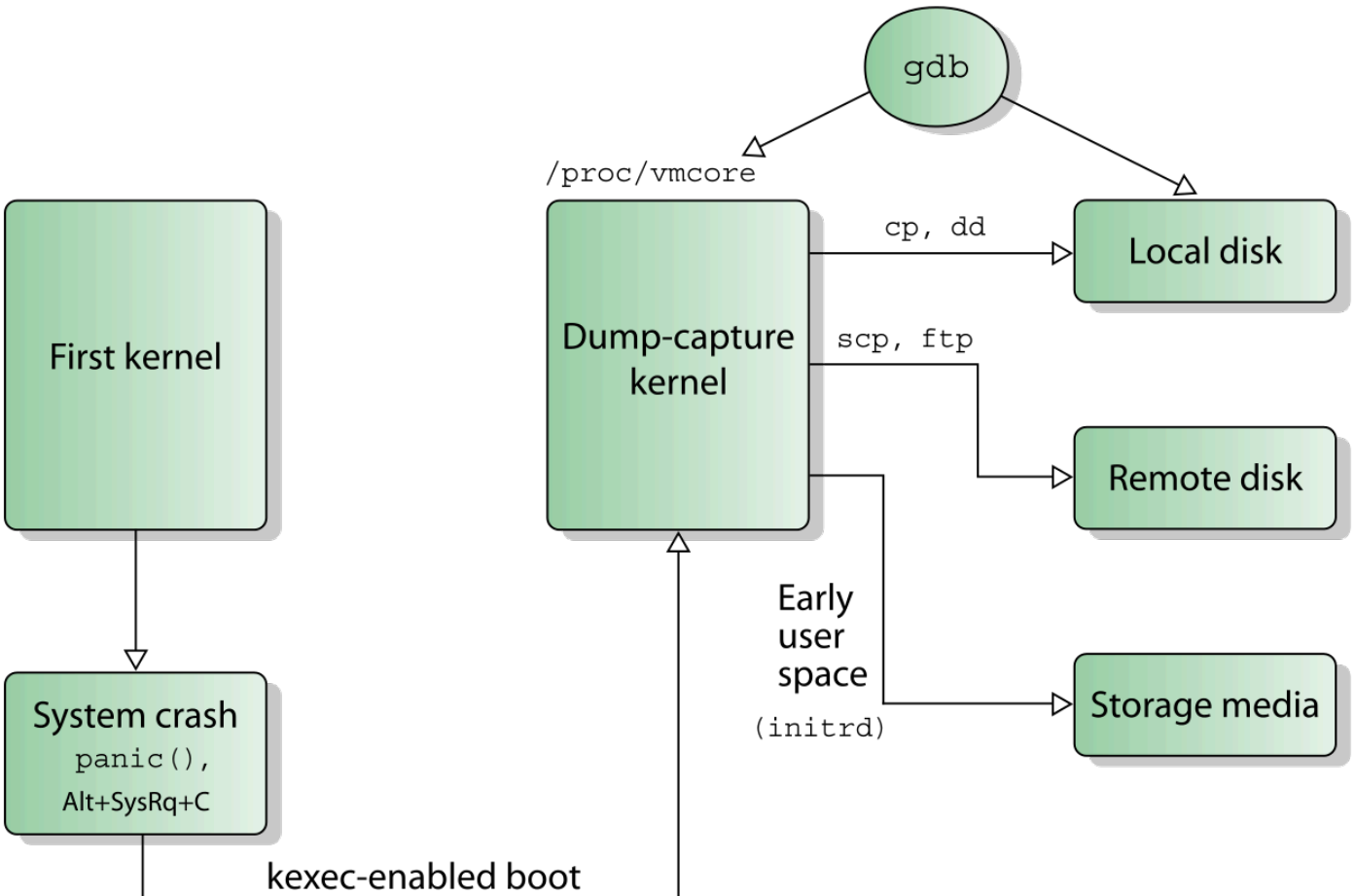


Image credits: Wikipedia



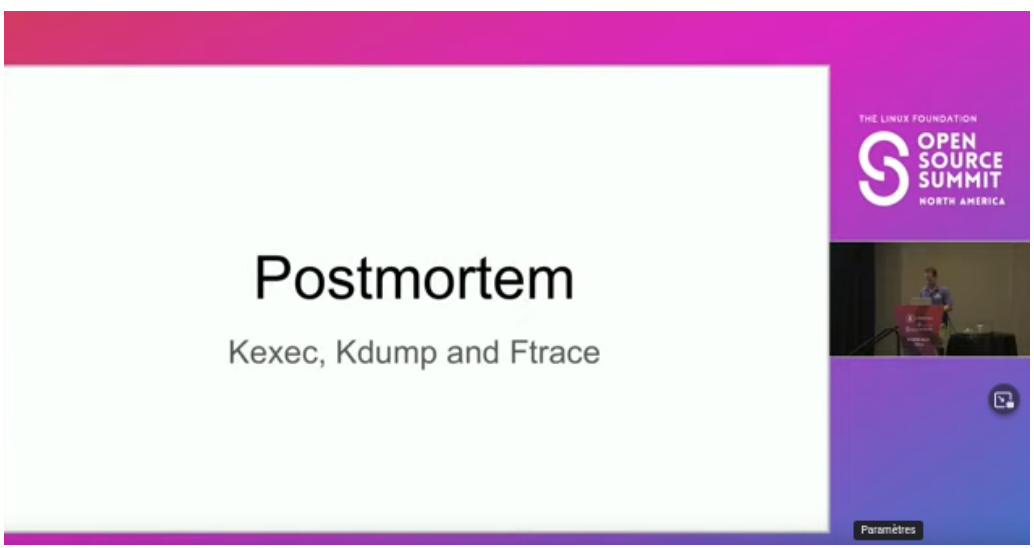
kexec config and setup

- On the standard kernel:
  - CONFIG_KEXEC=y to enable KEXEC support
  - `kexec-tools` to provide the `kexec` command
  - A kernel and a DTB accessible by `kexec`
- On the dump-capture kernel:
  - CONFIG_CRASH_DUMP=y to enable dumping a crashed kernel
  - CONFIG_PROC_VMCORE=y to enable `/proc/vmcore` support
  - CONFIG_AUTO_ZRELADDR=y on ARM32 platforms
- Set the correct `crashkernel` command line option:
  - `crashkernel=size[KMG][@offset[KMG]]`
- Load a dump-capture kernel on the first kernel with `kexec`:
  - ```
    kexec --type zImage -p my_zImage --dtb=my_dtb.dtb \
    --initrd=my_initrd --command-line="kernel command line"
    ```
- Then simply wait for a crash to happen!
    ```



Going further with kexec & kdump

- Presentation from Steven Rostedt about using kexec, kdump and ftrace with lot of tips and tricks about using kexec/kdump
  - Video: https://www.youtube.com/watch?v=aUGNDJPpUUg
  - Slides: https://static.sched.com/hosted_files/ossna2022/c0/Postmortem_%20Kexec%2C%20Kdump%20and%20Ftrace.pdf





pstore (1/3)

- Linux provides a filesystem interface for Persistent Storage (`pstore`) to save data across system resets: kernel logs, oopses, ftrace records, user messages...
- The platform needs to provide a persistent area to pstore (a block device, reserved RAM which is not reset on reboot, etc). Then you can enable a pstore frontend.
- ramoops is a common frontend for pstore: it will log any panic/oops to a pstore-managed ram buffer, which will be accessible on next boot
- Saved logs can be retrieved on next boot thanks to the pstore filesystem
- Some earlier software components in the boot chain (eg: U-Boot), if properly configured, may be able to access pstore data as well



pstore (2/3)

- Kernel configuration:
  - CONFIG_PSTORE=y
  - CONFIG_PSTORE_RAM=y
- Platform configuration: reserve some memory for pstore and configure it
  - Either through kernel command line:

```
mem=<usable_memory_size> ramoops.mem_address=0x8000000 ramoops.ecc=1
```

  - Or through device tree:

```
reserved-memory {
        [...]
        ramoops@8f000000 {
                compatible = "ramoops";
                reg = <0 0x8f000000 0 0x100000>;
                record-size = <0x4000>;
                console-size = <0x4000>;
        };
};
```



pstore (3/3)

- After a crash, the collected logs/traces will be available in the pstore filesystem:

```
mount -t pstore pstore /sys/fs/pstore
```

- If your data is not present in pstore filesystem, some services spawned early may have already collected/moved it
  - For example: systemd-pstore

Practical lab - Kernel debugging



Post-mortem debugging of a kernel crash

- Setup kexec, kdump and extract a kernel coredump